

실행환경 설명서

구분	내용
팀명/성명	금방
작품명	전북 금융 사각지대 진단과 찾아가는 은행 최적 동선 설계
실행 형태	☒ 로컬 실행 ☐ 웹 실행 ☐ 클라우드 실행 ☐ 시연영상 ☐ 기타
결과물 유형	☐ Notebook ☒ Python 파일 ☐ 웹앱 ☒ 대시보드 ☐ 챗봇 ☐ 기타

1. 개발 및 실행환경

항목	내용
운영체제	macOS 15 (Apple Silicon)에서 개발· 검증. Linux, Windows(WSL2 또는 Git Bash) 실행 가능
개발언어	Python 3.12 이상 (검증 3.14.6), JavaScript(ES2020)
주요 라이브러리	numpy 2.4.6, scipy 1.17.1, osmium 4.3.1 — `requirements.txt`세 개가 전부이고 나머지는 Python 표준 라이브러리. 브라우저 테스트는 Node.js 20 이상 (검증 26.4.0)의 내장 `node:test` 사용, 추가 패키지 설치 없음
사용 DB	없음. CSV· JSON 파일 기반
AI/분석 도구	SciPy Ward 계층 군집(분석 모델). 개발 보조로 Claude, Codex 사용 — 상세 범위는 [서식 7] 및 `docs/AI_USAGE.md`
실행 URL	해당 없음 — 로컬에서 `docs/demo-jeonbuk/index.html`을 열면 대시보드가 동작한다
GitHub URL	https://github.com/ekdms0502/2026-jeonbuk-finance-access 공개 저장소이며 제출 압축파일과 동일한 내용이다. 심사 시 이 저장소를 그대로 받아 실행하면 된다.

2. 실행 방법

단계	명령어 또는 설명
1단계	저장소를 받아 폴더로 이동한다. git clone https://github.com/ekdms0502/2026-jeonbuk-finance-access.git cd 2026-jeonbuk-finance-access (git이 없으면 저장소 페이지의 Code → Download ZIP 압축을 풀어 이동한다)
2단계	가상환경을 만들고 활성화한다. python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
3단계	라이브러리를 설치한다.

	python3 -m pip install -r requirements.txt
4단계	전체 검증을 실행한다. 1분 내외 걸리고, 마지막 줄에 PASS (offline)이 나오면 정상이다. bash verify_offline.sh
5단계	대시보드를 연다. 로컬 서버가 필요 없다. open docs/demo-jeonbuk/index.html (Windows: start docs\demo-jeonbuk\index.html)

3. 주요 파일 구성

파일명	설명
`README.md`	실행 방법, 실행 환경, 📁 📁 📁 , 결과 요약을 담은 실행 설명서
`requirements.txt`	라이브러리 목록 (numpy, scipy, osmium)
`verify_offline.sh`	주 실행 파일. 원본· API 키 없이 전체 파이프라인을 재실행하고 검증한다.
`src/`	수집· 정제· 분석· 경로· 시각화 코드 21개 모듈
`data/processed/`	코드가 읽는 가공 데이터 28개. 전량 포함되어 있어 바로 실행된다.
`data/raw/`	원본 데이터 17개 파일 포함 (osm.pbf 285MB만 Release 자산으로 별도 제공)
`data_sample/`	가공 데이터 상위 20행 미리보기와 데이터 설명서
`result/reports/`	분석 결과 리포트 4종 (품질· 커버리지· 유형평가· 시나리오 근거)
`result/outputs/`	최종 산출 CSV· JSON 사본
`docs/demo-jeonbuk/index.html`	대시보드· 지도 화면. 브라우저로 직접 연다.
`docs/demo-jeonbuk/methodology.html`	계산 수식· 군집· 경로 이론 설명 화면
`config/jeonbuk_route_default.json`	운영 가정과 입력 범위 설정
`tests/`	경로 엔진· 유형화· 대시보드 엔진 테스트
`docs/AI_USAGE.md`	AI· 바이트 코딩 활용 범위
`docs/THIRD_PARTY_NOTICES.md`	오픈소스 출처· 라이선스
`data/source_manifest.json`	원천별 URL· 기준일· 수집일· 라이선스· SHA-256 원장
`docs/verify_offline_log.txt`	검증 실행 로그 원문 (PASS (offline) 전 과정)
`reports/`	verify_offline.sh가 갱신하는 검증 리포트 출력 폴더

4. 실행 시 유의사항

- API 키가 없어도 된다. `verify\_offline.sh`와 대시보드는 네트워크 없이 동작한다.
- `.env`의 ECOS 키는 원본을 새로 수집할 때만 필요하고 제출물에 포함되지 않는다.
- `data/raw/` 원본 데이터도 저장소에 포함되어 있다. 단 `south-korea-latest.osm.pbf` (285MB)만 GitHub 파일당 100MB 제한 때문에 저장소 Release(raw-data-v1) 자산으로 제공하며, SHA-256이 `data/source\_manifest.json` 원장과 일치한다.
- 재다운로드에 필요한 URL·기준일·SHA-256은 `data/source\_manifest.json`에 전부 있다.
- 원본이 없어도 `verify\_offline.sh`는 전부 통과한다.
- `verify\_source\_lineage.sh` (원본 계보 검증)는 Release의 osm.pbf를 `data/raw/`에 내려받은 뒤 실행하면 75/75 전체 통과한다. pbf 없이 실행하면 부족한 파일 목록을 출력하고 중단되며, 이는 정상 동작이다. 일상 심사에는 `verify\_offline.sh`만으로 충분하다.
- Node.js가 없으면 3단계의 브라우저 엔진 테스트만 건너뛴다. Python 분석은 그대로 돈다.
- `osmium`설치가 실패해도 된다. 원본 OSM 도로망을 새로 만들 때만 쓴다.
- `numpy`와 `scipy`만 설치해도 `verify\_offline.sh`는 통과한다.
- 대시보드는 폴더째 열어야 한다. `docs/fonts/`가 함께 있어야 글꼴이 정상 표시된다.
- Windows에서 `.sh`실행이 안 되면 WSL2 또는 Git Bash를 쓴다.
- `verify.sh`는 API 키 보유 환경용 최소 검증이다. 심사 시에는 `verify\_offline.sh`만 실행하면 된다.
- 검증 기록: `bash verify\_offline.sh` 종료 코드 0, PASS (offline), 약 35초 소요. `bash verify\_source\_lineage.sh` (osm.pbf 배치 후) 75/75 checks PASS. 검증 환경 Python 3.14.6 / Node.js 26.4.0 / macOS 15 (Apple Silicon). 로그 원문은 `docs/verify\_offline\_log.txt`.